Using the AAMT Substrate: Integration Points and Wiring Status

Authors: Weslyn Cory Whitehead Jr.[1]

Affiliations: [1] AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: https://orcid.org/0009-0005-7707-3210

Submitted: July 2026
Working Paper Series: AAMT-WP-25
Anchor DOI: 10.5281/zenodo.19600795

Status: Working paper, not peer reviewed. Every claim in section 2's wiring-status table was checked directly against the source in this commit, not written from memory — the same audit anyone can re-run with the `grep` commands cited inline.

Abstract

WP-23 specifies what the `.aamt` substrate *is* — the binary format and the mechanisms built on it. WP-24 explains how the platform *is architected* and how that compares to other systems. Neither answers the question this paper exists to answer: **given all of that, what do I actually call, in which language, and is it running anywhere yet?** That last clause matters — a mechanism can be built, tested exhaustively, and exposed identically across four language bindings, and still not be called by a single production code path. This paper draws that line explicitly rather than letting "implemented" quietly mean two different things, and has been revised twice as that line moved: mirror pairing, RS(15,13)/GF(16), and the composed read path that resolves what to serve on an unrecoverable mismatch (WP-23 §13.1) are now **live**, wired into `memory_index_bridge.py`'s real routing path — the wiring itself surfaced a genuine bug (a discovery query composing raw candidates with `read_verified` breaks on a bucket that legitimately mixes primaries with unrelated mirrors), fixed with new native `queryVerified/ queryVerifiedSoft` functions rather than papered over in the binding layer. Soft routing gained real wrapper methods in the browser client and Unity this revision too (this paper's own first version inaccurately claimed Unity already had one — corrected in section 3.2 with what was actually checked, not assumed). As of this writing: five mechanisms are **live** — pack/query, soft routing, mirror+RS+composed-read, the Odu cross-session profile feeding WP-22's escape steering, and the lineage journal. Two are **library-ready** — CRDT convergence with persisted replica identity (WP-23 §13.2), and columnar bulk export, deliberately unused for the one workload it was tried against. One is **research-only** — echo refocusing. Section 2 is the audited table; section 3 is a working code snippet for each mechanism; section 4 is how to build each layer; section 5 is what remains for the mechanisms not yet live, stated as engineering decisions rather than "just call the function."

1. Orientation: the four places this substrate lives

`packages/libaamt` is the single source of truth. Everything else is a binding over the same C ABI (`packages/libaamt/include/aamt/aamt.h`), so a `.aamt` file packed by any one of these opens identically in the other three.

- **Core** — `packages/libaamt/src/*.{hpp,cpp}`. C++17, no external dependencies. Builds to `libaamt.{dylib,so,dll}` via CMake, and the same source compiles to WASM via Emscripten (`packages/libaamt/wasm/build.sh`).
- **Desktop (Python)** — `packages/libaamt/bindings/python/aamt_ffi.py` (ctypes), consumed today by `maiiam-alchemist/apps/desktop/scripts/memory_index_bridge.py` (the production bridge process), `odu_coverage_map.py`, and `wp21_rolling_output_prototype.py` (the rolling generation loop). `aamt_crdt.py` sits alongside as an independent module — CRDT merge logic operates on Python objects before anything is packed, so it has no C++ counterpart and needs none.
- **Browser (WASM)** — `packages/libaamt/wasm/dist/aamt.{mjs,wasm}`, wrapped by a small hand-written client, `aamt-client.js`, in the AAMT-Mathematics-Visualizer repository (a sibling repo, not

under `asamanthinks-platform`), deployed at `asamanthinks.com/alchemist/visualizer`.
- **Unity (C#)** — four files under `Assets/MaiiaM/Systems/Substrate/` (`AamtNative`, `AamtSubstrate`, `AamtSubstrateService`, `AamtSubstrateBridge`) in the MaiiaM Platform project — a separate repository, tracked mostly outside this git repo (see section 4) — P/Invoking the same native library.

2. Wiring status, audited

Mechanism	Exposed via	Live caller	Status
Pack + point query	all four layers	desktop, browser, Unity	**Live**
Soft routing	all four layers	desktop only	**Live**, Python only
Odu profile → escape prior	all four layers	desktop, full loop	**Live**, full loop
Lineage journal	C ABI + Python	desktop only	**Live**, Python only
Mirror pairing + RS, composed read (§13.1)	all four layers	desktop (`memory_index_bridge.py`)	**Live**
CRDT convergence + replica id + real 2nd writer (§13.2-13.3)	Python + browser JS	none	**Library-ready**
Columnar bulk export	all four layers	none in production	**Library-ready**, deliberately unused
Echo refocusing	standalone script	none	**Research-only**

Source and live-caller detail for each row, in order:

1. **Pack + query** — `core.{hpp,cpp}` `Builder::add`/`Reader::query`; called by `memory_index_bridge.py` (`add`/`build`/`query`/`route`), the visualizer's `aamt-client.js` `query()`, Unity's `AamtSubstrateService.Route()`.
2. **Soft routing** — `core.cpp` `Reader::routeSoftTopK`; called by `memory_index_bridge.py`'s `route` command with `soft=true`. Correction to this paper's own first revision: it previously claimed Unity's `AamtNative` already bound `aamt_query_soft` — checked directly against the source for this revision, it did not; there was no P/Invoke declaration at all, only the C ABI and Python binding existed. Fixed: `AamtNative.aamt_query_soft`, `AamtSubstrate.QuerySoft`, `AamtSubstrateService.RouteSoft` (Unity) and `AamtSubstrate.querySoft` (the browser client, `aamt-client.js`) are now real, tested wrapper methods — verified live in the deployed visualizer's dev server, rank-0 branch checked to equal `query()`'s hard route on a real coordinate against the shipped demo substrate. Still not called by any default UI flow in either the browser demo or a Unity scene — the gap closed is "no wrapper existed," not "nothing invokes it yet."
3. **Odu profile** — `core.cpp` `Reader::oduProfile`; `memory_index_bridge.py`'s `profile` command feeds `OduCoverageMap.seed_persistent()`, called from `wp21_rolling_output_prototype.py`'s rolling-generation loop — the only row where every hop in the chain is real.
4. **Lineage journal** — `core.cpp` journal write path, Python's `AamtJournal`/`read_journal`/`verify_journal`; `memory_index_bridge.py` opens one per session, logs every `route()`, exposes `verify_journal` as its own wire command. No WASM or Unity journal writer exists yet.
5. **Mirror pairing + RS + composed read** — `core.cpp` `Builder::addMirroredWithECC`, `Reader::readVerified`/ `Reader::queryVerified`/`Reader::queryVerifiedSoft` (WP-23 §13.1); full parity across all four layers as of this revision — Unity's `AamtSubstrate.QueryVerified`/`QueryVerifiedSoft` and `AamtSubstrateService.RouteVerified`/`RouteVerifiedSoft`, and the browser client's `queryVerified`/`queryVerifiedSoft` (verified live against the shipped demo substrate: raw `query()` and `queryVerified()` agree on a clean file, soft rank-0 matches the hard route). Unity is unverified by a live compiler (none available this session); the browser client's `uint64_t`-by-value `payload_capacity` parameter needed a real JS `BigInt` (Emscripten's `WASM_BIGINT` default since ~3.1.51) rather than a plain Number — caught by testing live, not assumed. `memory_index_bridge.py` packs every record with `add_mirrored_ecc` and every `route()`/`route(soft)` candidate goes through `query_verified`/`query_verified_soft` before it leaves the process. Verified against a real RS silent-miscorrection (a genuine two-nibble collision found by exhaustive search, not simulated)

injected into a real packed index: the corrupted candidate is dropped from `route()`'s output while `bucket_size` still reports the true raw count, so the drop is visible rather than silent.

6. **CRDT** — `aamt_crdt.py`, standalone by design (WP-23 §12.1: no C++ counterpart needed); `local_replica_id` (WP-23 §13.2) closes the replica-identity provisioning gap.
7. **Columnar bulk export** — `core.cpp Reader::allRecordsColumnar`, Unity's `AllRecordsBulk`; used only in `examples//`benchmark scripts. WP-23 corrections ledger #8 found it loses to `json.load` for `memory_index_bridge.py`'s actual label-table workload, so the bridge kept labels in JSON — not an oversight, a measured decision.
8. **Echo refocusing** — `packages/libaamt/experiments/echo_refocus.py`, a standalone Monte Carlo script, not a callable API.

Reproduce the still-true "none" callers yourself: `grep -rn "aamt_crdt" maiiam-alchemist/apps/desktop/scripts/*.py` returns nothing — CRDT is the one redundancy mechanism still unwired, honestly, because no second writer exists yet to make wiring it meaningful (§5.3).

3. How to call each mechanism

3.1 Pack + query (the baseline every other row extends)

```python
import sys
sys.path.insert(0, "packages/libaamt/bindings/python")
import aamt_ffi

with aamt_ffi.AamtBuilder(max_depth=8, bucket_capacity=32) as b:
    b.add(id_=42, tera=[0.2, 0.7, 0.5, 0.9], data=b"hello substrate")
    b.write("out.aamt")

with aamt_ffi.AamtSubstrate.open("out.aamt") as sub:
    result = sub.query([0.2, 0.7, 0.5, 0.9], capacity=16)
```

Unity (reader only — Unity does not pack files in this platform, it opens one placed in `StreamingAssets` at startup):

```
var sub = AamtSubstrate.Open(path);
var result = sub.Query(new TeraVector(0.2f, 0.7f, 0.5f, 0.9f));
```

Browser: `new AamtSubstrate(Module).query([0.2, 0.7, 0.5, 0.9])` after loading `aamt.mjs` and fetching the `.aamt` bytes into its virtual FS.

3.2 Soft routing

```
branches = sub.query_soft(tera, k=16, per_branch_capacity=8)
# ranked list of all 16 root Meji orthants by product-Bernoulli weight,
# rank 0 == the hard route by construction (WP-23 corrections #10)
```

Wired at the bridge: send `{"cmd":"route","args":{"tera":[...],"k":8,"soft":true}}` to `memory_index_bridge.py`. The browser client and Unity now have wrapper methods too:

```
const branches = sub.querySoft([0.2, 0.7, 0.5, 0.9], 16, 8);
```

```
var branches = AamtSubstrateService.Instance.RouteSoft(tera, k: 16, perBranchCapacity: 8);
```

Neither is called by a default UI flow yet — the browser demo and Unity scenes still only exercise the hard route — but the wrapper gap this paper's first revision flagged (and inaccurately claimed was already closed on the Unity side) is closed.

3.3 Odu profile → escape steering (the fullest live loop)

This is the one mechanism worth tracing end to end, because it is the only one where every layer is real: `memory_index_bridge.py`'s `profile` command calls `Reader::oduProfile()`, which returns per-cell record counts and TERA centroids across all 256 Odu cells; `wp21_rolling_output_prototype.py` passes that straight into `OduCoverageMap.seed_persistent(counts, centroids)`, which is what closes the WP-22

escape-steering gap identified in WP-23 §9 — a session now starts with a real prior over where the substrate's history actually lives, not a blank one.

```
# desktop bridge, one round trip
res = store.profile(aamt_path)                # -> {"counts": [...256], "centroids": [...256*4]}
ocm.seed_persistent(res["counts"], res["centroids"])
```

3.4 Lineage journal

```
with aamt_ffi.AamtSubstrate.open(aamt_path) as sub, \
     aamt_ffi.AamtJournal(journal_path) as journal:
    result = sub.query(tera)
    journal.log(sub, tera, result)

result = aamt_ffi.verify_journal(aamt_path, journal_path)
assert result.ok
```

Wired at the bridge (`memory_index_bridge.py` opens a journal per session automatically and logs every `route()`); `verify_journal` is its own wire command. Python-only — no WASM or Unity journal writer exists, so a browser or Unity session cannot currently produce a replayable `.aamtj`.

3.5 Mirror pairing + RS + composed read — live

```
with aamt_ffi.AamtBuilder(8, 32) as b:
    b.add_mirrored_ecc(id_=42, tera=[0.2, 0.7, 0.5, 0.9], data=b"both defenses")
    b.write("out.aamt")

with aamt_ffi.AamtSubstrate.open("out.aamt") as sub:
    r = sub.read_verified(id_=42, tera_hint=[0.2, 0.7, 0.5, 0.9])
    if r.ok:
        use(r.data)          # r.status tells you which tier resolved it
    else:
        pass                 # AAMT_READ_UNAVAILABLE - refused, not guessed

    # discovery path (don't already know the id): query_verified filters to
    # primaries only and verifies each - this is what route() actually calls
    discovered = sub.query_verified(tera=[0.2, 0.7, 0.5, 0.9], capacity=8)
    for rec in discovered.records:
        use(rec.data)
```

RS(15,13) is tried first (cheap, mirror untouched on a clean or single-nibble-repaired read); the mirror is consulted only when RS is unresolvable or the caller wants the independent check; a mirror that positively decodes to *different* bytes than the primary is the only case refused outright (WP-23 §13.1). `query_verified`/`query_verified_soft` exist as separate discovery-path functions rather than composing `read_verified` over raw `query()` hits in the binding layer, because a bucket can legitimately mix primaries with unrelated mirrors and Python has no visibility into record flags to filter them — this is the exact bug found while wiring `memory_index_bridge.py` (below), fixed natively in `core.cpp` rather than patched around in `aamt_ffi.py`.

Live: `memory_index_bridge.py` packs every record with `add_mirrored_ecc` and both `route()` and `route(soft)` verify every candidate through `query_verified`/`query_verified_soft` before returning it — a candidate the mirror positively disputes is dropped, and `bucket_size` still reports the true raw count so the drop is visible. Full C ABI/Python/WASM parity; Unity has the C ABI declarations and dylib but no C# wrapper yet for the verified-read functions specifically (Unity is a reader-only client in this platform, and its existing `ReadVerified` wrapper predates `query_verified`/`query_verified_soft` — the single-id read path has Unity parity, the discovery path does not yet).

Stateless RS codec, if only the cheap layer is wanted without mirroring:

```
encoded = aamt_ffi.encode_ecc_payload(b"raw bytes")
result = aamt_ffi.decode_ecc_payload(encoded)  # EccDecodeResult(data=..., corrected_symbols=...)
```

Remember the measured ceiling before reaching for this alone (WP-23 §11): a second corrupted nibble in the same 13-nibble block is silently mis-corrected 86.7% of the time — exactly what `read_verified`/`query_verified` exist to catch by pairing it with the mirror.

3.6 CRDT convergence — library-ready, now with a real second writer

```
from aamt_crdt import HLC, CrdtStore, local_replica_id

clock = HLC.zero(replica_id=local_replica_id("/path/to/this/install/state")).tick()
store = CrdtStore()
store.put(42, [0.2, 0.7, 0.5, 0.9], b"written offline", clock)

converged = store.merge(other_replicas_store)    # commutative, associative, idempotent
converged.to_aamt("converged.aamt")              # normal, immutable pack of the converged state
```

No C++ counterpart by design — see WP-23 §12.1: the substrate stays immutable-once-packed, and convergence is an orchestration question that belongs above that boundary, not inside it. `local_replica_id` (WP-23 §13.2) closed the identity-collision gap; `aamt-crdt.js` (WP-23 §13.3, AAMT-Mathematics-Visualizer repo) is a real second writer — a browser tab, not a mock, with its own persisted replica identity and the exact same HLC/tiebreak/JSON-interchange semantics, verified to merge correctly with a real Python-side store including a genuine conflict:

```
import { HLC, CrdtStore, localReplicaId } from './aamt-crdt.js';
const clock = HLC.zero(localReplicaId()).tick();
const store = new CrdtStore();
store.put(42, [0.2, 0.7, 0.5, 0.9], new TextEncoder().encode('written in the browser'), clock);
const json = store.toJson(); // hand this to a Python process's CrdtStore.from_json
```

Still **not live** in the WP-25 sense stated precisely: no shipped consumer (`memory_index_bridge.py`, the visualizer's demo page) actually generates cross-device writes for this to converge today. What changed is narrower — the second writer now exists and interoperates, proven, not just specified — so wiring this in is no longer blocked on "there is nothing real to test this against."

3.7 Echo refocusing — research-only

```
python3 packages/libaamt/experiments/echo_refocus.py
```

This runs the Monte Carlo validation from EXP-02, not a callable API — there is nothing to "use" here yet. The precondition (coherent vs. incoherent drift) is proven on a synthetic model; EXP-02 §6 names the measurement (autocorrelation on real `OduCoverageMap` visit sequences) required before this could graduate to library-ready.

4. Building each layer

Core + tests.

```
cd packages/libaamt && cmake -B build && cmake --build build && ctest --test-dir build
```

Runs `roundtrip.cpp` (the main integration suite — mirror-pairing and ECC corruption-injection tests live here) and `test_rs_gf16.cpp` (the exhaustive GF(16) proof).

Python bindings. No build step — `aamt_ffi.py` loads the built `libaamt.{dylib,so}` via ctypes at import time (`lib_path` argument, or the platform-default search path). Property tests: `python3 packages/libaamt/examples/test_crdt.py`, `python3 packages/libaamt/examples/test_journal.py`.

WASM.

```
brew install emscripten   # or: source emsdk_env.sh
packages/libaamt/wasm/build.sh
```

Fixed 128 MB memory, growth disabled — WP-23 corrections #9's Safari `TextDecoder` fix. Output goes to `packages/libaamt/wasm/dist/`; copy `aamt.mjs`/`aamt.wasm` into the visualizer's `public/wasm/` to update the live demo.

Unity. Build `libaamt.dylib`/`.so`/`.dll` from the core step above for each target platform and drop it in `Assets/Plugins/<platform>/`. On Apple Silicon the dylib is ad-hoc signed and runs with no extra codesigning step. `AAMTBootstrap.cs` (not tracked in this git repo — see the Unity project's own Plastic SCM history) is what actually instantiates `AamtSubstrateService` in a running scene.

5. What's left for the mechanisms not yet live

Mirror pairing and RS/GF(16) are no longer "if you want to wire this in" — section 3.5 covers what's actually running. What follows is what remains for CRDT and columnar bulk export, and it's worth recording how the mirror+RS integration actually resolved its own open question, since it's the template for both: `memory_index_bridge.py` switched from `b.add(...)` to `b.add_mirrored_ecc(...)` and from `sub.query(...)` to `sub.query_verified(...)`/`query_verified_soft(...)` — the "what happens on an unrecoverable payload mismatch" question resolved to "refuse rather than guess, and let `bucket_size` still show the true raw count so a drop is visible, never silent" — and the per-record cost question (worth doubling file size + RS's ~15/13 expansion for *every* record?) resolved to yes specifically *because* the substrate here is a derived, always-rebuildable routing cache with cheap string payloads — a decision that would come out differently for a substrate storing large or expensive-to-regenerate payloads, which is exactly why section 3.5 states it as a decision made for this consumer rather than a universal default.

5.1 CRDT — identity closed, a real second writer exists, wiring it into the shipped product is what's left (WP-23 §13.2-13.3). The gate this row used to be behind — "needs an actual second writer to be worth wiring in at all" — is gone: `aamt-crdt.js` is a real browser-side writer, verified to interoperate with the Python side including a genuine conflict, not a hypothetical. What's left is narrower and still real: `memory_index_bridge.py` is a single-process, single-writer bridge today, and nothing in the shipped visualizer demo generates a second device's writes either — wiring CRDT in means picking an actual place a second writer's output would come from (a companion app, a second desktop session, the visualizer itself accumulating local edits) and building *that*, not the convergence layer, which already works. Building the convergence layer into a still-single-writer bridge today would still be building for a hypothetical (see WP-23's own restraint about not adding parity symbols beyond what RS's measured ceiling needs) — but "no second writer exists to test against" is no longer the reason to wait.

5.2 Columnar bulk export. Already tried for `memory_index_bridge.py`'s label table and lost to `json.load` (WP-23 corrections #8) — not a "wire it in" candidate for that workload. It remains the right primitive for Unity and WASM bulk reads, which have no competing `json.load` advantage to lose to; the decision there is simply whether any Unity/WASM consumer needs a bulk read yet, not measured demand today.

Reference implementation: `packages/libaamt`. Format spec: WP-23. Architecture and positioning: WP-24. Physics validation: EXP-01, EXP-02. This paper supersedes nothing in those three — it is the map from what they prove to what actually runs.